

COMP9334 Computing clusters

1 Probability distribution

1.1 The probability distribution of inter-arrival time

The value of each inter-arrival time is obtained by multiplying an exponentially distributed random number with a uniformly distributed random number. The implementation of Python3 code is shown in Figure 1.1. Here, 10000 generation tests are conducted using the parameters provided in test case 4, and a distribution histogram of the generated data is plotted, as shown in the Figure 1.2.

```
def generate_interarrival(self):  
    a1 = random.expovariate(self.lambd)  
    a2 = random.uniform(self.alpha2l, self.alpha2u)  
    return a1 * a2
```

Figure 1.1

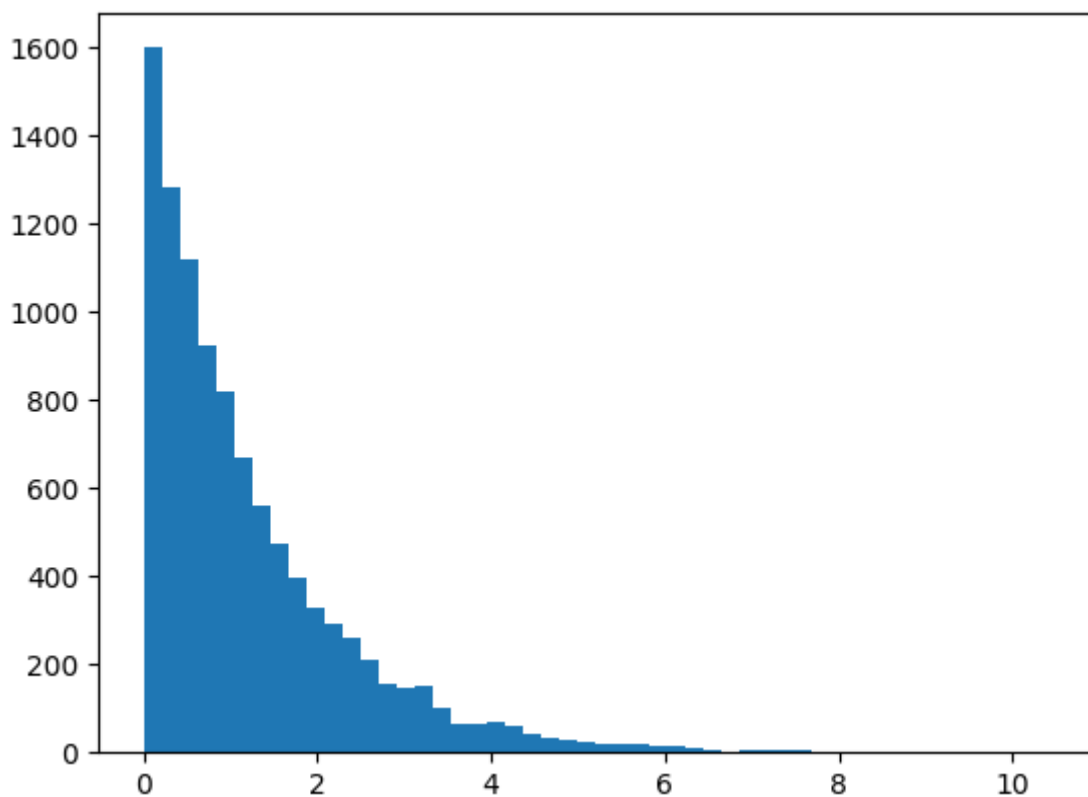


Figure 1.2

1.2 The probability of service group

The test case provides a p_0 parameter, which determines the probability of group being 0, so the probability of group to 1 is $(1-p_0)$. Here, the p_0 value from test case 4 was used for testing, with a total of 10000 iterations. Finally, the proportion of group 0 was calculated, as shown in Figure 1.3. Therefore, the result is 0.6967, which is almost the same as the probability of 0.7 specified in test case 4.

```
1 count = 0
2 for group in group_list:
3     if group == 0:
4         count += 1
5 print('group 0 / (group 0 + group 1) = ', count / len(group_list))
✓ 0.0s
group 0 / (group 0 + group 1) = 0.6967
```

Figure 1.3

1.3 The probability distribution of service time

I used the Inverse Transform Method here to generate a service time that conforms to the given probability density function(PDF). We mainly rely on the SymPy module in Python 3 to derive the Cumulative Density Function(CDF), and further derive the inverse function(ICDF). The specific code implementation and derivation results of the inverse distribution functions of g_0 and g_1 are shown in Figures 1.4 and 1.5.

$$\gamma_0 = \alpha_0^{-\eta_0} - \beta_0^{-\eta_0}$$
$$g_0(t) = \begin{cases} 0 & \text{for } 0 \leq t \leq \alpha_0 \\ \frac{\eta_0}{\gamma_0 t^{\eta_0+1}} & \text{for } \alpha_0 < t < \beta_0 \\ 0 & \text{for } t \geq \beta_0 \end{cases}$$

```
1 t, alpha0, beta0, eta0, alpha1, eta1, x = sy.symbols('t alpha0 beta0 eta0 alpha1 eta1 x')
2 gamma0 = alpha0**(-eta0) - beta0**(-eta0)
3 g0_pdf = eta0 / (gamma0 * t**(eta0 + 1))
4 g0_cdf = sy.integrate(g0_pdf, (t, alpha0, t))
5 print('CDF:')
6 display(g0_cdf)
```

CDF:

$$\begin{cases} \frac{t^{-\eta_0}}{-\beta_0^{-\eta_0} + \alpha_0^{-\eta_0}} + \frac{\alpha_0^{-\eta_0}}{-\beta_0^{-\eta_0} + \alpha_0^{-\eta_0}} & \text{for } \eta_0 > -\infty \wedge \eta_0 < \infty \wedge \eta_0 \neq 0 \\ -\frac{\eta_0 \log(\alpha_0)}{-\beta_0^{-\eta_0} + \alpha_0^{-\eta_0}} + \frac{\eta_0 \log(t)}{-\beta_0^{-\eta_0} + \alpha_0^{-\eta_0}} & \text{otherwise} \end{cases}$$

```
1 g0_inverse_cdf = sy.solve(sy.Eq(g0_cdf, x), t)
2 print('ICDF')
3 display(g0_inverse_cdf[1])
```

ICDF

$$\begin{cases} \left(\frac{\alpha_0^{-\eta_0} \beta_0^{\eta_0}}{\alpha_0^{-\eta_0} - \beta_0^{-\eta_0}} x + \beta_0^{-\eta_0} \right)^{\frac{1}{\eta_0}} & \text{for } \eta_0 > -\infty \wedge \eta_0 < \infty \wedge \eta_0 \neq 0 \\ \text{NaN} & \text{otherwise} \end{cases}$$

Figure 1.4



Figure 1.5

At this point, generating service time transforms into generating a random number with a uniform distribution of 0 to 1. The generated random number is substituted into the value obtained by ICDF, which is a random number that conforms to the probability distribution function. The code implementation is shown in Figure 1.6.

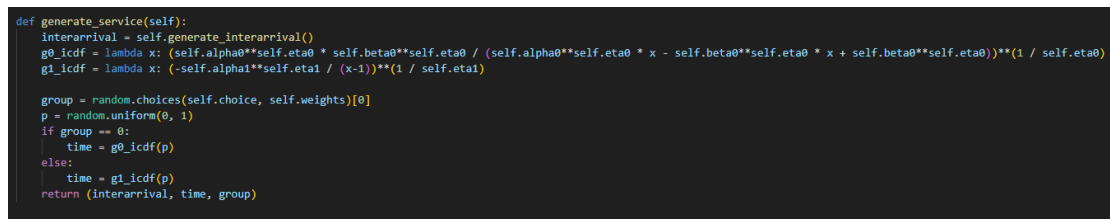


Figure 1.6

Based on the parameters provided in test case 4, generate 10000 times and draw the distribution histograms of service time generated by group 0 and group 1, respectively, as shown in Figures 1.7 and 1.8.

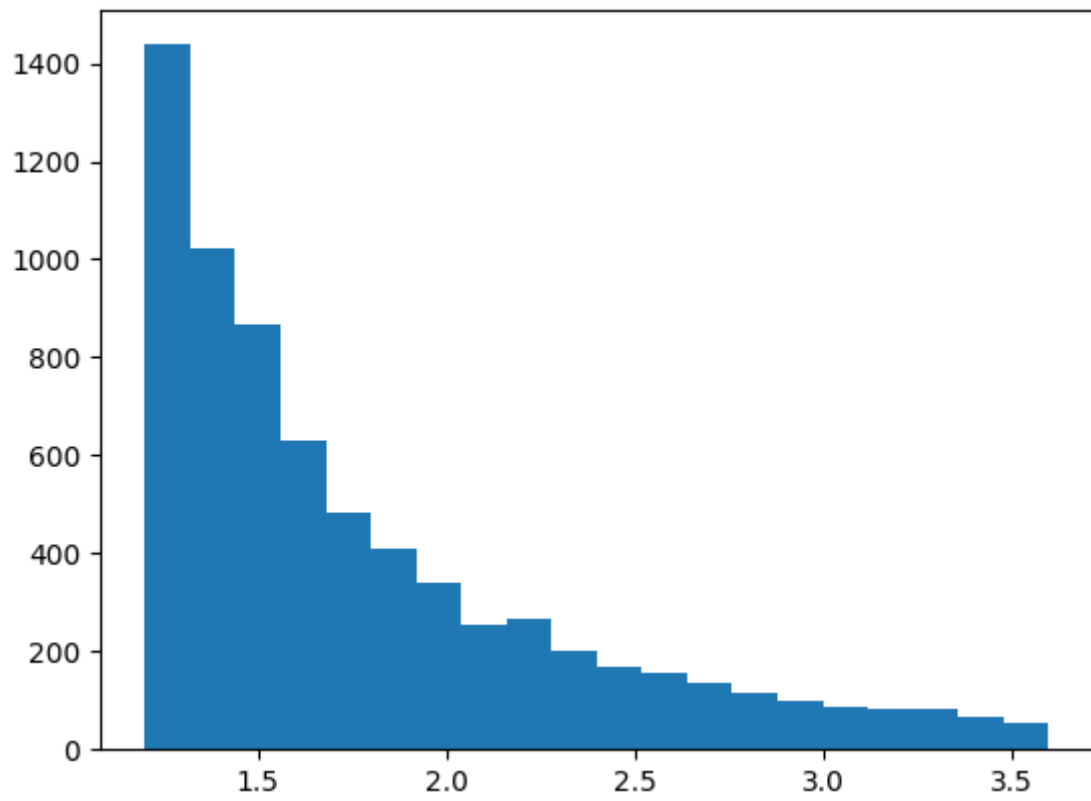


Figure 1.7

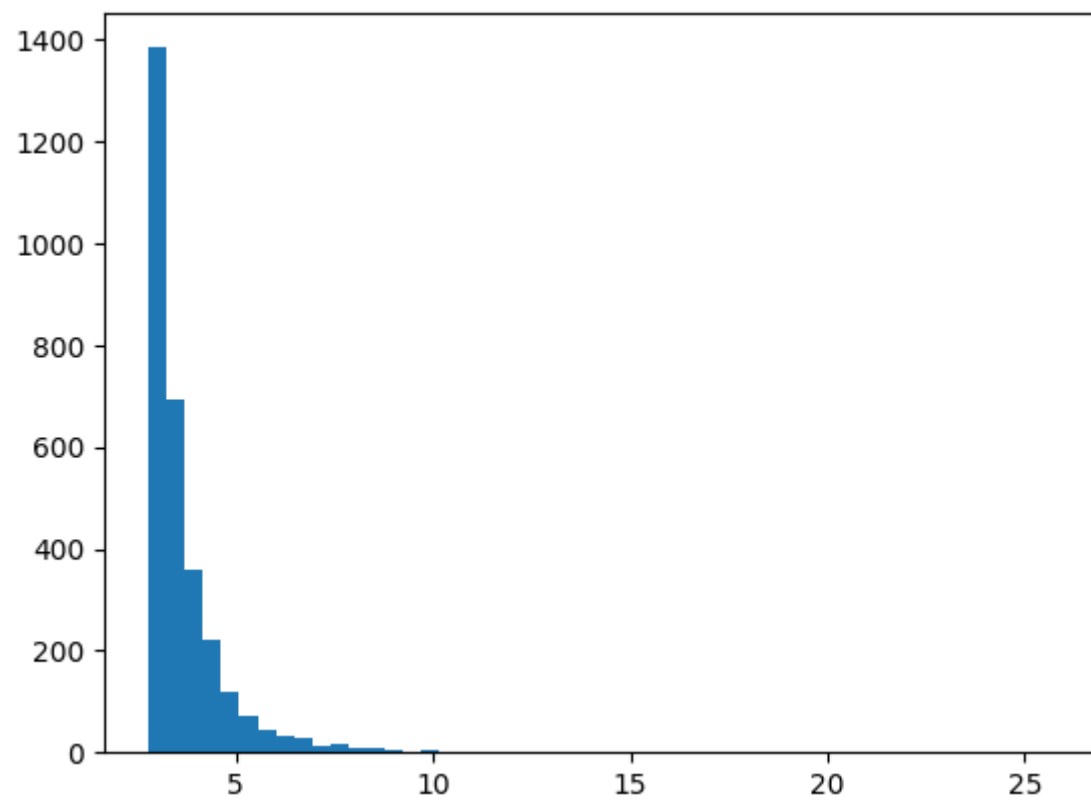


Figure 1.8

2 Verifying the correctness of the simulation program

In order to verify the correctness of the simulation program, I used 7 test cases provided by lecturer to simulate and run them separately. Finally, I used `cf_output_with_ref.py` for verification, and the verification output results are shown in the following Figure 2.1.

```
1  !sh ./run_test.sh 0
2  !sh ./run_test.sh 1
3  !sh ./run_test.sh 2
4  !sh ./run_test.sh 3
5  !sh ./run_test.sh 4
6  !sh ./run_test.sh 5
7  !sh ./run_test.sh 6
8
9  !python3 ./cf_output_with_ref.py 0
10 !python3 ./cf_output_with_ref.py 1
11 !python3 ./cf_output_with_ref.py 2
12 !python3 ./cf_output_with_ref.py 3
13 !python3 ./cf_output_with_ref.py 4
14 !python3 ./cf_output_with_ref.py 5
15 !python3 ./cf_output_with_ref.py 6

✓ 22.1s

Test 0: Mean response time matches the reference
Test 0: Departure times match the reference
Test 1: Mean response time matches the reference
Test 1: Departure times match the reference
Test 2: Mean response time matches the reference
Test 2: Departure times match the reference
Test 3: Mean response time matches the reference
Test 3: Departure times match the reference
Test 4: Mean response time is within tolerance
Test 4: Mean response time is within tolerance
Test 5: Mean response time is within tolerance
Test 5: Mean response time is within tolerance
Test 6: Mean response time is within tolerance
Test 6: Mean response time is within tolerance
```

Figure 2.1

3 Simulation reproducible

To ensure that my simulation experiment is reproducible, I used a fixed random number method. Here, the parameters of test case 4 will be used for 100 repeated simulation tests, and the mean response time for each experiment will be recorded. Draw an image based on the mean response time of each experiment as shown in the figure 3.1.

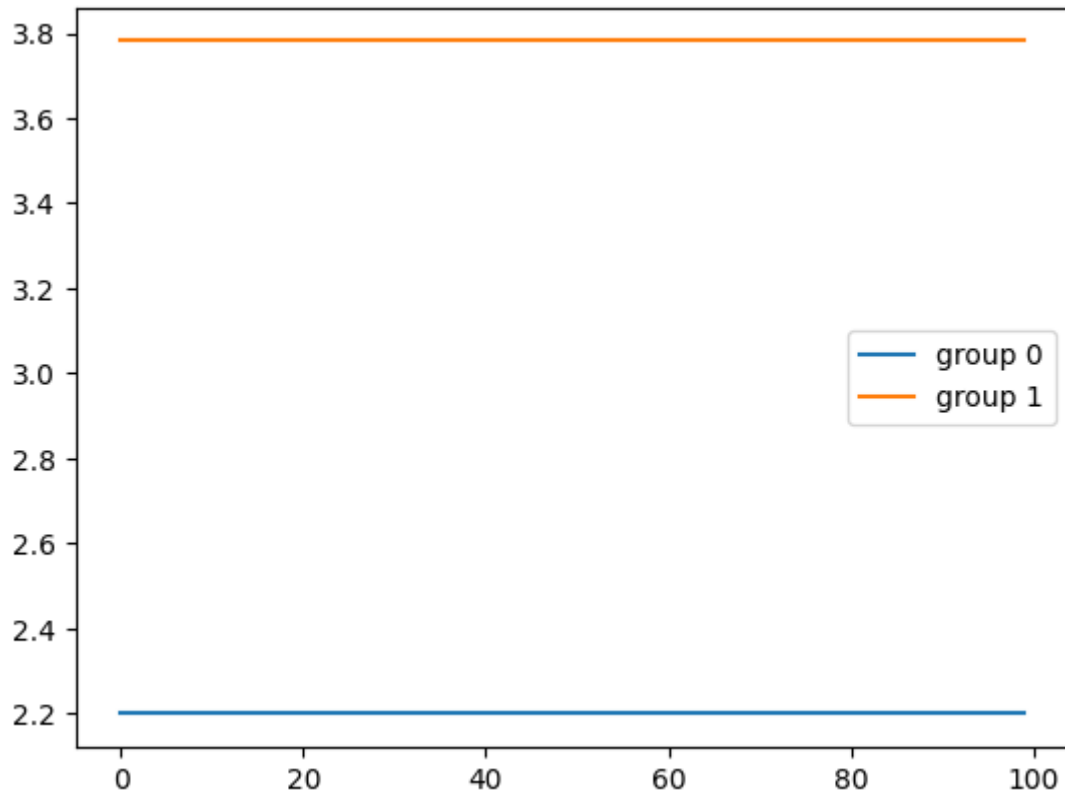


Figure 3.1

4 Determine a suitable value of n_0

4.1 Running the simulation once and present the results

The main purpose of simulation is to determine the optimal value of n_0 , and to achieve this goal, it is necessary to ensure that the data is reliable. Here, we will mainly test using the parameters provided by test case 4, and set `time_end` to 10000, `n` to 10, and n_0 to 5, and then investigate transient behavior.

Set `time_end` to 10000 here to ensure that there is enough data to exhibit steady behavior. After conducting simulation tests, it was found that both group 0 and group 1 can maintain a steady mean response time after 1300 data points, as shown in the figure 4.1 below. Based on the probabilities of group 0 and group occurrence being 0.7 and 0.3, respectively (test case 4), the estimated minimum `time_end` value for steady behavior occurrence is 4333. The `time_end` value used here is 10000, which can provide sufficient data for the experiment.

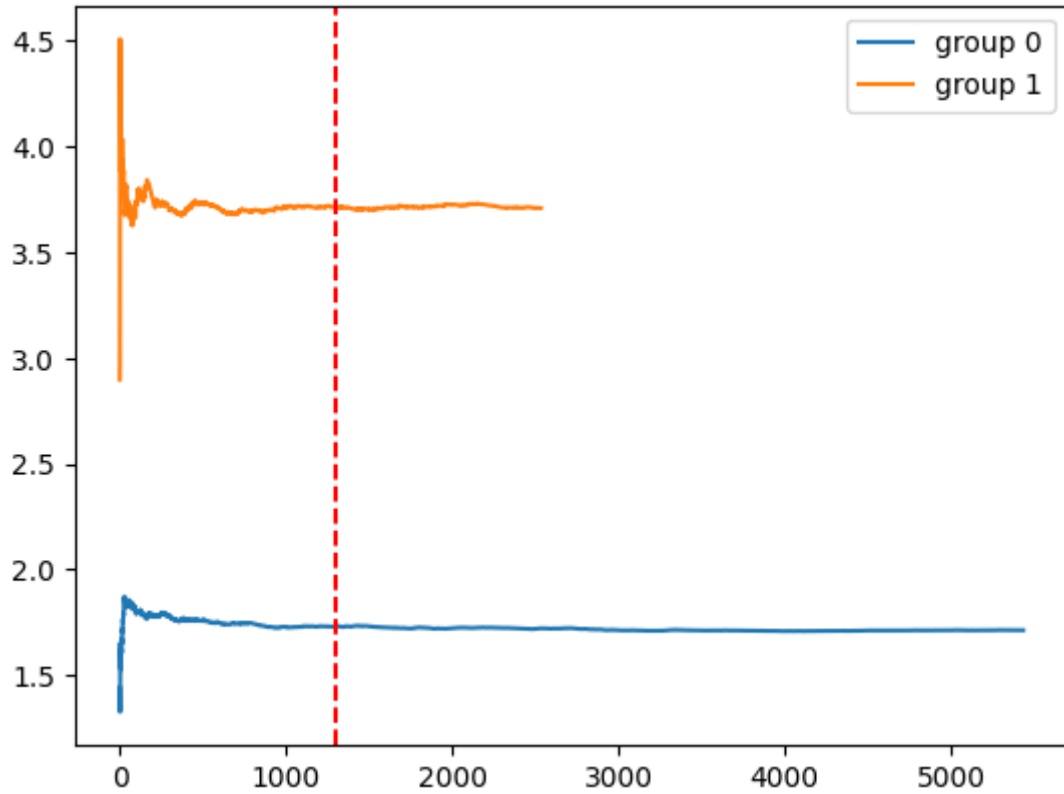


Figure 4.1

In further experiments, the first 1300 data points for group 0 and group 1 will be removed, and the mean response time will be calculated. Directly calculating the mean response times of group 0 and group 1 are 1.7131 and 3.7074, respectively. After removing the transient, the calculated average values are 1.7073 and 3.7029, respectively.

4.2 Take different n_0 value and compare different systems

To determine the optimal value of n_0 , we need to calculate confidence intervals separately. Firstly, we need to calculate the sample mean using the following formula,

$$\hat{T} = \frac{\sum_{i=1}^n T(i)}{n}$$

The code implementation is shown in the figure 4.2,

```
1 def mean(data):
2     return sum(data) / len(data)
```

Figure 4.2

Then the sample standard deviation is,

$$\hat{S} = \sqrt{\frac{\sum_{i=1}^n (\hat{T} - T(i))^2}{n-1}}$$

The code implementation is shown in the figure 4.3,

```

1 def sample_standard_deviation(x, mean_value):
2     S = 0
3     for i in range(len(x)):
4         S += (mean_value - x[i])**2
5     return ( S / ( len(x) - 1 ) )**0.5

```

Figure 4.3

There is a probability 95% that the mean response time that I want to estimate lies in the area of

$$\left[\hat{T} - t_{n-1, 1-\frac{\alpha}{2}} \frac{\hat{S}}{\sqrt{n}}, \hat{T} + t_{n-1, 1-\frac{\alpha}{2}} \frac{\hat{S}}{\sqrt{n}} \right]$$

The code implementation is shown in the figure 4.4,

```

1 def confidence_interval(ssd, m):
2     lower_value = '{:.4f}'.format(m - 2 * ssd / sampling_frequency**0.5)
3     upper_value = '{:.4f}'.format(m + 2 * ssd / sampling_frequency**0.5)
4     return (lower_value, upper_value)

```

Figure 4.4

In the experiment, two mean response times, group 0 and group 1, will be generated. The lecturer provides the following formula for calculation,

$$\text{weighted_mean_response_time} = \omega_0 T_0 + \omega_1 T_1$$

w_i is a ratio, the numerator is the proportion of the i -th (0,1) group in the total amount of the two groups, and the denominator representing the average service time of the i -th group.

After each simulation program execution, two sets of response times and the service time of the task itself are written to the file. Then read and calculate the values of w_0 and w_1 corresponding to each simulation data, calculate the mean response time for both sets, and then calculate the weighted mean response time, as shown in Figure 4.5,


```

1 import subprocess
2
3 modify_time_end(time_end)
4 for n0 in range(1, n):
5     modify_n0(n0)
6     with open('n0_{}.wmrt.txt'.format(n0), 'w') as wmrt:
7         for seed in range(1001, 1001 + sampling_frequency):
8             print('n0={}, seed={}'.format(n0, seed))
9             rt0_list = []
10            rt1_list = []
11            t0_list = []
12            t1_list = []
13            modify_seed(seed)
14            subprocess.run('sh ./run_test.sh test', shell=True)
15            with open('response_time_group0_test.txt', 'r') as f:
16                for line in f:
17                    rt, t = line.strip().split()
18                    rt0_list.append(float(rt))
19                    t0_list.append(float(t))
20            with open('response_time_group1_test.txt', 'r') as f:
21                for line in f:
22                    rt, t = line.strip().split()
23                    rt1_list.append(float(rt))
24                    t1_list.append(float(t))
25            rt0_list = rt0_list[start:]
26            rt1_list = rt1_list[start:]
27            t0_list = t0_list[start:]
28            t1_list = t1_list[start:]
29            group0_percent = len(rt0_list) / (len(rt0_list) + len(rt1_list))
30            group1_percent = 1 - group0_percent
31            w0 = group0_percent / mean(t0_list)
32            w1 = group1_percent / mean(t1_list)
33            wmrt.write('{}\n'.format(mean(rt0_list) * w0 + mean(rt1_list) * w1))

```

Figure 4.5

In order to minimize the error of the simulation experiment as much as possible, we will adopt a method of repeating the experiment multiple times. At the same time, we need to consider the reproducibility of the experimental site, so we will use multiple fixed random seeds for the experiment. When n is set to 10, n_0 will test values from 1 to 9. After each value of n_0 , 20 repeated experiments will be conducted, with random seed values ranging from 1001 to 1020. Finally, the confidence interval of the data obtained from simulating the 9 values of n_0 is calculated, as shown in Figure 4.6,

```

1 print('n0\t\t\tconfidence interval')
2 for n0 in range(1, n):
3     wmrt_list = []
4     with open('n0_{}.wmrt.txt'.format(n0), 'r') as f:
5         wmrt_list = [float(wmrt_value.strip()) for wmrt_value in f]
6     mwrt = mean(wmrt_list)
7     ssd = sample_standard_deviation(wmrt_list, mwrt)
8     ci = confidence_interval(ssd, mwrt)
9     print('{}\t\t\t{}'.format(n0, ci))

```

✓ 0.0s

n0	confidence interval
1	('98.0171', '98.8670')
2	('1.1653', '1.1669')
3	('1.0249', '1.0251')
4	('1.0041', '1.0042')
5	('1.0008', '1.0008')
6	('1.0011', '1.0012')
7	('1.0060', '1.0064')
8	('1.0409', '1.0421')
9	('6.3308', '9.2929')

Figure 4.6

The confidence interval data obtained after completing the simulation test are shown in Table 4.1.

n ₀	Confidence Interval
1	(98.0171,98.8670)
2	(1.1653,1.1669)
3	(1.0249,1.0251)
4	(1.0041,1.0042)
5	(1.0008,1.0008)
6	(1.0011,1.0012)
7	(1.0060,1.0064)
8	(1.0409,1.0421)
9	(6.3308,9.2929)

Table 4.1

It can be observed that when $n_0 = 5$, the minimum confidence interval can be obtained.